

The Abstractions Are Slipping: Four OS Mismatches in the Agent Era

Zheng Liu Qin Long Jinli Zhang Yong Yang Tao Ma

Alibaba Cloud Computing

Hangzhou, China

{wenqing.lz, zhuolong.lq, jinli.zjl, zhiche.yy, boyu.mt}@alibaba-inc.com

Abstract

AI agents are now a major cloud workload, running on an operating system built for processes that execute deterministic instructions. We name four mismatches and support them with a failure and two witnesses. We also report a negative result of our own: speculative execution returned 4–11% of wall-clock time on the class of tasks it suited best, while the draft model that made the predictions costs 2–20% of a frontier model on every turn whether it hits or misses, so the mechanism does not pay for itself. DeepSeek’s harness ships a hand-written scheduler, a journal, and a Landlock sandbox, yet cannot express whether a tool call may be retried, so after a crash it asks the model to decide. Omarchy launches ten agent programs and hands eight of them that vendor’s own flag for switching permission prompts off. The turn boundary and the side-effect class are facts no current abstraction can carry, and we argue they belong below the agent.

1 Introduction

An AI agent runs in a loop: the model emits a tool call, something executes it, the result returns, and it decides again. Where a process is a continuous instruction stream, an agent is a sequence of turns, each a short burst of compute around a long wait.

Revisiting OS abstractions when the workload changes is an old move: Stonebraker on databases in 1981 [23], Engler and Kaashoek on ill-fitting abstractions in 1995 [8], the Berkeley View on machine learning in 2017 [22]. Agents are the next case, not because they use GPUs, which training already forced, but because the shape of their execution is new.

A deployed agent is three components: a *harness* driving the loop, an *LLM runtime* serving tokens, and a *tool sandbox* executing actions. Naming which component a problem sits in separates an OS problem from a framework problem. M1 and M3 arise inside one machine, between the harness and the kernel beneath it. M2 arises at the harness–sandbox boundary, which both sides cross through the kernel. M4 reaches past both into third-party services the kernel does not control, which is why it needs a declared contract rather than inspection.

IBM Research argues the same case from the cloud side, and notes independently that skills carry no formal interface contract [21]. Their primitives sit in the orchestration layer; we argue two of these contracts must reach the syscall boundary, because a policy above it is bypassable by the agent it constrains.

Two of our own measurements are released with this paper: a deployed agent on one production instance, 347 sessions and 2,682 model calls over three weeks, and a controlled benchmark of 30 sessions and 180 turns over five tool types used only for the per-tool breakdown. Every number below comes from one of the two.

2 Four Mismatches

M1: Turns versus the process model. Across those 347 sessions the agent spent 8.7% of elapsed time in inference and 4.2% in tool calls; for the other 87% the session was open and it was doing nothing, one gap running 92 hours. Within the active time inference took 67.6% and tools 32.4%, but per session that split ran from 0.7% to 82%: the share outside inference is a property of the workload, not a constant.

An agent idle seven-eighths of the time is a scheduling opportunity the kernel cannot see: it observes a process holding memory, with nothing to distinguish a task between turns, cheap to displace, from one mid-turn. Our runtime sorts session interruptions into a dozen kinds [1], of which only the crash is a failure the OS recognises; a truncated Server-Sent Events stream reaches the kernel as a socket closing normally. Nothing marks a process as an agent either, so our observer must guess: it matches command lines, then resolves symbols and attaches probes to the TLS library, which takes 8–15 s, and an agent that issues its first request sooner is never observed. DeepSeek’s harness makes turns first-class with durable boundary events, then rebuilds a scheduler above the kernel to act on them [5]. We concede burst-and-wait is not unique to agents, and that turns form a graph, not a chain. We also tried to remove the boundary rather than describe it: with inference reaching 1000+ tokens per second [24, 28], a draft model predicts the next tool call and the runtime runs it during generation, keeping the result if correct. Local development was hopeless at a 5–6% hit rate, because read and incremental builds finish in milliseconds; on web tasks whose URLs were given in the prompt, seven of ten returned 4–11%. It still does not pay for itself, because the draft model is a fixed cost: 2–20% of a frontier model on every turn, hit or miss, the same band as the returns. Nor would a better hit rate help much: each turn’s generation waits on the previous result, and pages fetched early enlarge the next turn’s context. Better models will not remove the layer, as scaling has not fixed long-horizon failure modes [16, 25].

M2: Tool contracts in the framework, not the system. Tool output is written for humans: on our own corpus 26–78% of a tool response is droppable overhead. Agents are not stuck reading untyped bytes: DeepSeek’s harness mandates a canonical output schema per tool and validates every result against it [5], and typed interfaces are long established [3, 7]. What survives is narrower, and the OS has absorbed this shape of problem before. A dependency that only the build system knows does not survive the binary travelling, so ELF’s DT_NEEDED puts that fact in the artifact and ld.so enforces it at load [14]. A format and a loader absorbed the contract; the tools were never asked to agree. Agents are a new kind of caller and their contract is still at the convention stage: the type lives

in the framework, so it binds only what that framework wraps, and the same tool invoked from a shell carries nothing. That is not ecosystem mess; it is what slippage looks like early.

M3: State at five timescales versus the file system. Agent state must be managed at five lifetimes at once, from a decode step to a months-long project: the attention key-value (KV) cache [19, 30], working context, episodic memory, persistent knowledge, and archive. In our own deployment session lifetimes ran from a median of 7 s to a longest of 3.9 days. No single system defines all five tiers; we assemble them from work that each builds part of the hierarchy [4, 13, 18, 29]. The tiers have different owners and cannot see each other: the cache is keyed by prefix and blind to agent identity, episodic memory is keyed by session and blind to whether the cache still holds that prefix. Resume an evicted agent and the cache is gone while the memory is intact, with nothing reconciling them. In DeepSeek’s harness, cache semantics are instead carried in a prose section in every package’s README, which the maintainers declined to generate from source because static analysis cannot infer the conditions [5]. The timescales come from the outside world: a larger context window moves items between tiers, it does not collapse them.

M4: Branching versus linear execution. Agents take actions whose outcomes they cannot predict, so they must try, inspect, and undo. This is not sampling randomness: a deterministic policy still cannot know whether a build succeeds until it runs. Mechanisms are cheap now: 10.83 ms checkpoints [6], forks of filesystem, process and GPU state [12, 26, 27], and test-time scaling makes rollouts routine [20]. The gap is everything outside local state, so we separate actions by how far they can be undone: pure-local, local-reversible, remote-idempotent, remote-reversible, irreversible-external. The kernel plainly cannot infer these levels for itself: a socket write may be a query, an email, or a payment. It does not have to. It needs only to know whether an action can be undone, which the tool author knows when they write the tool. DeepSeek’s harness is one proof that this is structural. It mandates output schemas but cannot declare a tool read-only, idempotent, or reversible, so when a crash leaves an outcome unrecorded its recovery path injects a prompt into the model’s context: retry only if the operation is read-only or idempotent, otherwise verify external state or ask the user [5]. A correctness decision a type system could settle is handed to a language model as prose, because no type exists to hold it.

Our own speculation study reached it independently, treating read-only or idempotent as a precondition for pre-execution rather than a tuning parameter. Omarchy shows the other end. This distribution treats coding agents as first-class desktop citizens. It launches ten of them, and for eight it passes that vendor’s own flag for switching permission prompts off, its code observing that each needs its own spelling of “don’t stop to ask” and that the remaining two offer none to skip; a passwordless-sudo helper names AI agents in its warning text [2]. Eight spellings of the same escape hatch, and no way at all to say the narrower thing, is what is left when there is nothing to declare.

3 Four Directions

Almost all of this is achievable above the kernel: turn-aware scheduling, tiered memory, and copy-on-write forking, the last

in Crab [27], and ours is built that way too [1], on bubblewrap, seccomp, and a privileged snapshot daemon. Two things resist it. A user-space policy makes the agent both the subject of the constraint and the process enforcing it, so under prompt injection the trust relation collapses. And a harness mediates a tool entry point while the effect lands at a syscall: an agent that switches SDK, or forks a subprocess running `curl` against a credential file, is outside that path entirely. Isolation is the practical consequence, and we paid for it: our first speculative runtime ran draft calls on the live request event loop, delaying reads of the real response enough to void a round of measurements. Speculative work must be distinguishable from committed work *to the scheduler*, and an irreversible action needs a gate the agent cannot route around.

D1: Make the agent a first-class kernel entity. AIOS schedules agents from user space [17] and Continuum works inside the inference layer [15]; neither can tell the CPU which thread is speculative. A `TASK_AGENT` flag on `task_struct`, a `turn_boundary()` call marking one edge of the turn graph, and reclamation that prefers between-turn tasks would be enough to start, and the flag alone removes the discovery race above.

D2: Give tool contracts a place to live. MCP standardises how a tool describes itself [11] but does not bind the tool. A tool should carry its response schema the way an ELF object carries `DT_NEEDED`, and the OS should check it at invocation the way `ld.so` checks at load, returning a typed error that names the offending field. Our shell `cosh` interposes on a tool call through a pre-invocation hook [1], which is the right check in the one place an agent can step around.

D3: One hierarchy across the five tiers. Make the tiers mountable behind one interface, as a VFS does for filesystems, so that a cache invalidation and a memory update can be ordered against each other: mechanism in the kernel, policy in the framework that knows what a memory means.

D4: Declared side effects, enforced at the syscall boundary. Existing forks cover GUI state [12], containers [27], or stay in the framework [9]. Declare the five levels at registration beside the schema; the OS then gates speculation on the declaration and inspects no traffic, and anything undeclared counts as irreversible. Sagas made this trade in 1987 [10], labelling an effect that cannot be made atomic with the action that compensates for it; we want that label at the syscall boundary, granting that rollback has hazards of its own [31].

4 Call to Action

That is what belongs below the agent, yet every project we examined rebuilds it above: lifecycle, snapshotting, sandboxing, memory management. Two agreements would start the work, and neither needs a new kernel: a way for a harness to signal a turn boundary, and a way for a tool to declare whether its effects can be undone. Both are worth standardising rather than reimplementing again.

References

- [1] ANOLISA Contributors. 2026. ANOLISA: An Operating System for AI Agents. Open-source software. Components cited here: `src/agentsight` (eBPF observability and interruption detection), `src/copilot-shell` (`cosh`), `src/ws-ckpt`

- (btrfs snapshots), src/agent-sec-core (OS-level policy). <https://github.com/alibaba/anolisa>
- [2] Basecamp. 2026. Omarchy. Open-source software. Version 4.0.0-alpha, commit 83881e9; source verified 2026-08-28. The per-agent permission flags are in bin/omarchy-agent; the sudo warning is bin/omarchy-sudo-passwordless:40-41. <https://github.com/basecamp/omarchy>
 - [3] Andrew D. Birrell and Bruce Jay Nelson. 1984. Implementing Remote Procedure Calls. *ACM Transactions on Computer Systems* 2, 1 (1984), 39–59. doi:10.1145/2080.357392
 - [4] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413 [cs.AI] <https://arxiv.org/abs/2504.19413>
 - [5] DeepSeek. 2026. DeepSeek Harness. Open-source software. Commit cd5ef81 (dsh-0.1.2-alpha.1); source verified 2026-08-28. The crash-recovery string cited here is in packages/core/session/src/repair.ts:104, documented at packages/core/session/README.md:141 and asserted in packages/core/session/tests/repair.spec.ts:259. <https://github.com/deepseek-ai/deepseek-harness>
 - [6] Yungpeng Dong, Jingkai He, Shiqi Liu, Yuze Hou, Dong Du, Zhonghu Xu, Si Yu, Baochuan Yang, Yubin Xia, and Haibo Chen. 2026. DeltaBox: Scaling Stateful AI Agents with Millisecond-Level Sandbox Checkpoint and Restore. arXiv:2605.22781 [cs.OS] <https://arxiv.org/abs/2605.22781>
 - [7] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2024. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. arXiv:2411.15100 [cs.CL] <https://arxiv.org/abs/2411.15100>
 - [8] Dawson R. Engler and M. Frans Kaashoek. 1995. Exterminate All Operating System Abstractions. In *Proceedings of the 5th Workshop on Hot Topics in Operating Systems (HotOS-V)*. IEEE Computer Society, 78–83. doi:10.1109/HOTOS.1995.513462
 - [9] Faisal Fareed. 2026. Cost-Aware Speculative Execution for LLM-Agent Workflows: An Integrated Five-Dimension Method. arXiv:2606.07846 [cs.AI] <https://arxiv.org/abs/2606.07846>
 - [10] Hector Garcia-Molina and Kenneth Salem. 1987. Sagas. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. ACM, 249–259. doi:10.1145/38713.38742
 - [11] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv:2503.23278 [cs.CR] <https://arxiv.org/abs/2503.23278>
 - [12] Yutong Huang, Vikranth Srivatsa, Alex Asch, Hansin Tushar Patwa, and Yiyang Zhang. 2026. TClone: Low-Latency Forking of Live GUI Environments for Computer-Use Agents. arXiv:2605.17320 [cs.OS] <https://arxiv.org/abs/2605.17320>
 - [13] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory OS of AI Agent. arXiv:2506.06326 [cs.AI] <https://arxiv.org/abs/2506.06326>
 - [14] John R. Levine. 1999. *Linkers and Loaders*. Morgan Kaufmann.
 - [15] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2026. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. In *Proceedings of the International Conference on Learning Representations (ICLR)*. doi:10.48550/arXiv.2511.02230
 - [16] Miles McCain, Thomas Millar, Saffron Huang, Jake Eaton, Kunal Handa, Michael Stern, Alex Tamkin, Matt Kearney, Esin Durmus, Judy Shen, Jerry Hong, Brian Calvert, Jun Shern Chan, Francesco Mosconi, David Saunders, Tyler Neylon, Gabriel Nicholas, Sarah Pollack, Jack Clark, and Deep Ganguli. 2026. Measuring AI Agent Autonomy in Practice. Anthropic Research. Published 2026-02-18. <https://www.anthropic.com/research/measuring-agent-autonomy>
 - [17] Kai Mei, Xi Zhu, Wujiang Xu, Wenyue Hua, Mingyu Jin, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. 2025. AIOS: LLM Agent Operating System. In *Proceedings of the 1st Conference on Language Modeling (COLM)*. doi:10.48550/arXiv.2403.16971
 - [18] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. In *Proceedings of the International Conference on Learning Representations (ICLR)*. doi:10.48550/arXiv.2310.08560
 - [19] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: A KVCache-Centric Disaggregated Architecture for LLM Serving. In *Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association. doi:10.1145/3773772
 - [20] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. arXiv:2408.03314 [cs.LG] <https://arxiv.org/abs/2408.03314>
 - [21] Malgorzata Steinder and Hubertus Franke. 2026. Towards an Agent Operating System: Lessons from Classical and Cloud OS. arXiv:2607.25076 [cs.OS] <https://arxiv.org/abs/2607.25076>
 - [22] Ion Stoica, Dawn Song, Raluca Ada Popa, David Patterson, Michael W. Mahoney, Randy Katz, Anthony D. Joseph, Michael Jordan, Joseph M. Hellerstein, Joseph E. Gonzalez, Ken Goldberg, Ali Ghodsi, David Culler, and Pieter Abbeel. 2017. A Berkeley View of Systems Challenges for AI. arXiv:1712.05855 [cs.AI] <https://arxiv.org/abs/1712.05855>
 - [23] Michael Stonebraker. 1981. Operating System Support for Database Management. *Commun. ACM* 24, 7 (1981), 412–418. doi:10.1145/358699.358703
 - [24] TileRT Team. 2026. Speed as the Next Scaling Law. TileRT Blog. Published 2026-05-21. <https://www.tileart.ai/blog/speed-as-the-next-scaling-law.html>
 - [25] Xinyu Jessica Wang, Haoyue Bai, Yiyu Sun, Haorui Wang, Shuibai Zhang, Wenjie Hu, Mya Schroder, Bilge Mutlu, Dawn Song, and Robert D. Nowak. 2026. The Long-Horizon Task Mirage? Diagnosing Where and Why Agentic Systems Break. arXiv:2604.11978 [cs.AI] <https://arxiv.org/abs/2604.11978>
 - [26] Xingda Wei, Zhuobin Huang, Tianle Sun, Yingyi Hao, Rong Chen, Mingcong Han, Jinyu Gu, and Haibo Chen. 2025. PhoenixOS: Concurrent OS-level GPU Checkpoint and Restore with Validated Speculation. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*. ACM. doi:10.1145/3731569.3764813
 - [27] Tianyuan Wu, Chaokun Chang, Lunxi Cao, Wei Gao, and Wei Wang. 2026. Crab: A Semantics-Aware Checkpoint/Restore Runtime for Agent Sandboxes. arXiv:2604.28138 [cs.OS] <https://arxiv.org/abs/2604.28138>
 - [28] Xiaomi MiMo Team. 2026. MiMo-V2.5-Pro-UltraSpeed: Pushing 1T-Parameter Model Generation Speed to 1000 TPS. Xiaomi MiMo Blog. Published 2026-06-08. <https://mimo.xiaomi.com/blog/mimo-tilert-1000tps>
 - [29] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110 [cs.AI] <https://arxiv.org/abs/2502.12110>
 - [30] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the 20th European Conference on Computer Systems (EuroSys)*. ACM. Best Paper Award. doi:10.1145/3689031.3696098
 - [31] Yusheng Zheng, Yiwei Yang, Wei Zhang, and Andi Quinn. 2026. ACRFence: Preventing Semantic Rollback Attacks in Agent Checkpoint-Restore. In *Proceedings of the Workshop on Co-Designing AI and Machine Infrastructure (CoDAIM), co-located with ASPLOS*. doi:10.48550/arXiv.2603.20625